

Sprawozdanie z projektu: Kompresja danych z użyciem SSN

Wykonawcy:
Radosław Ciepał
Kacper Mazurek
Krzysztof Kowalik

Prowadzący: dr inż. Piotr Kowalski
23 maja 2026

Spis treści

1	Wstęp i opis zadania	3
2	Preprocesing danych	3
3	Architektura modelu i implementacja	3
3.1	Enkoder i Dekoder	3
3.2	Kluczowe fragmenty kodu	4
4	Wyniki eksperymentów i analiza zależności	4
4.1	Zestawienie ilościowe	4
4.2	Wnioski i interpretacja	5
5	Instrukcja replikacji wyników	6

1 Wstęp i opis zadania

Celem niniejszego projektu było zaprojektowanie oraz implementacja wielowarstwowej sztucznej sieci neuronowej (SSN) typu Perceptron Wielowarstwowy (MLP) działającej w architekturze autoenkodera. Zadaniem systemu jest efektywna kompresja oraz dekompresja sygnałów elektrokardiograficznych (EKG).

Eksperymenty zostały przeprowadzone przy użyciu powszechnie uznanego zbioru danych *MIT-BIH Arrhythmia Dataset*. Każda pojedyncza próbka w zbiorze danych reprezentuje fragment przebiegu EKG i składa się ze 187 cech (wartości próbek sygnału w czasie). Głównym celem badawczym projektu było zbadanie bezpośredniej zależności zachodzącej pomiędzy stopniem kompresji (rozmiarem warstwy ukrytej) a jakością odzyskanego sygnału mierzoną za pomocą błędu średniokwadratowego (MSE).

2 Preprocessing danych

Właściwe przygotowanie danych wejściowych miało kluczowy wpływ na stabilność procesu uczenia oraz minimalizację błędu rekonstrukcji. Preprocessing składał się z następujących kroków:

1. **Usunięcie etykiet klasowych:** Ponieważ zadanie kompresji realizowane jest w sposób beznadzorczy (ang. *unsupervised learning*), ostatnia kolumna zbioru zawierająca klasyfikację chorób została odrzucona.
2. **Normalizacja danych:** Wartości sygnału zostały przeskalowane do przedziału $[0, 1]$ przy użyciu algorytmu `MinMaxScaler`. Parametry skalowania wyznaczono na zbiorze treningowym i zaaplikowano do zbioru testowego.
3. **Konwersja i Wsadowość:** Tablice danych przekształcono na tensory PyTorch oraz spakowano w obiekty `DataLoader` z rozmiarem paczki (batch size) ustawionym na 32.

3 Architektura modelu i implementacja

Zaimplementowany autoenkoder charakteryzuje się elastyczną architekturą typu *bottle-neck*. Układ warstw pośrednich jest generowany dynamicznie na podstawie zadanej zmiennej wymiaru ukrytego (`latent_dim`).

3.1 Enkoder i Dekoder

Enkoder redukuje wymiar wejściowy (187) do przestrzeni ukrytej, przechodząc przez warstwy liniowe o rozmiarach redukowanych proporcjonalnie (schodkowo przy użyciu potęg dwójki). Każda warstwa ukryta wykorzystuje normalizację wsadową (`BatchNorm1d`) oraz nowoczesną funkcję aktywacji `GELU`.

Dekoder odwraca ten proces. Na wyjściu dekodera zastosowano funkcję aktywacji `Sigmoid`, co zapewnia, że zrekonstruowane próbki ściśle odpowiadają zakresowi $[0, 1]$ wyznaczonemu w fazie preprocessingu.

3.2 Kluczowe fragmenty kodu

Poniższy listing przedstawia autorski algorytm automatycznego i symetrycznego generowania warstw autoenkodera na podstawie wybranej wielkości kompresji.

```
1 schodki = []
2 obecny_rozmiar = 128
3 while obecny_rozmiar > latent_dim:
4     schodki.append(obecny_rozmiar)
5     obecny_rozmiar = obecny_rozmiar // 2
6
7 # Budowanie Enkodera
8 encoder_siec = []
9 wejscie = 187
10 for wyjscie in schodki:
11     encoder_siec.extend([
12         nn.Linear(wejscie, wyjscie),
13         nn.BatchNorm1d(wyjscie),
14         nn.GELU()
15     ])
16     wejscie = wyjscie
17 encoder_siec.extend([nn.Linear(wejscie, latent_dim), nn.GELU()]])
```

Listing 1: Algorytm dynamicznego budowania warstw modelu (training.py)

4 Wyniki eksperymentów i analiza zależności

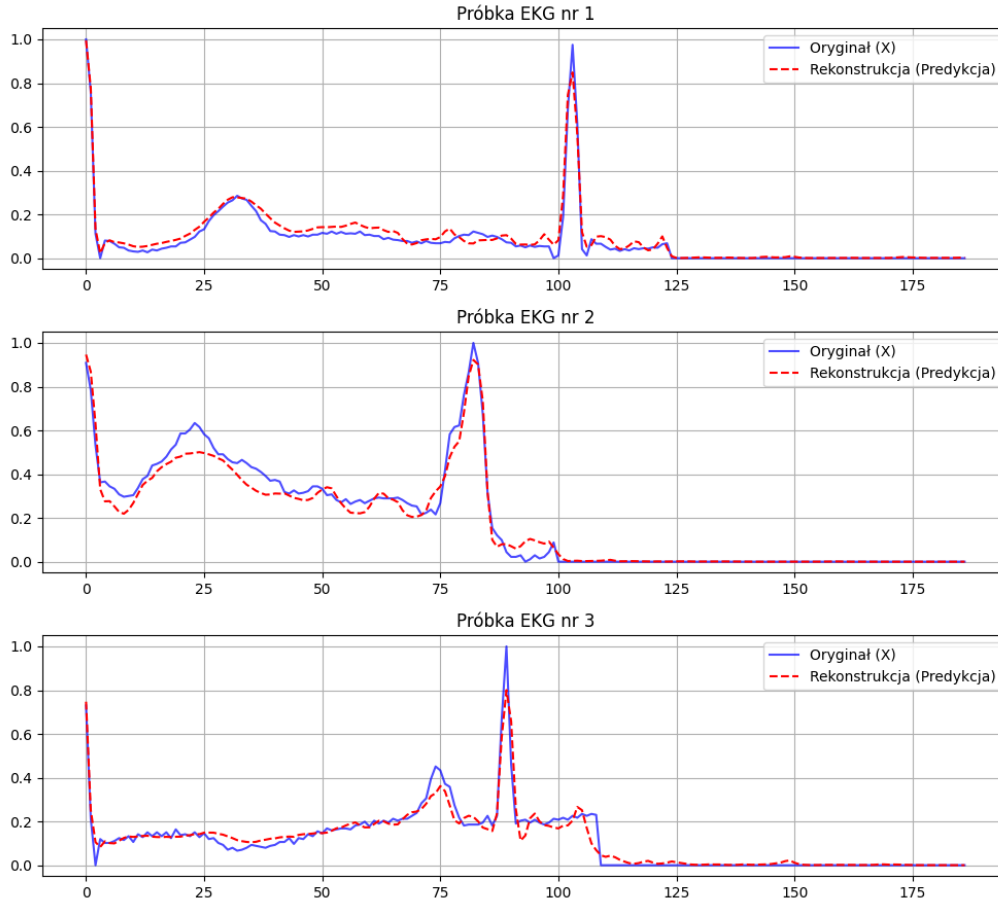
W celu zbadania zależności pomiędzy stopniem kompresji a wiernością rekonstrukcji przebiegu EKG, model został przetestowany dla różnych wymiarów przestrzeni ukrytej: 16, 40, 100 oraz 128. Trening każdego wariantu trwał określoną liczbę epok przy stałym współczynniku uczenia wynoszącym 10^{-3} z użyciem optymalizatora Adam.

4.1 Zestawienie ilościowe

W tabeli 1 przedstawiono przykładowe, uzyskane eksperymentalnie wartości błędu średniokwadratowego (MSE Loss) na zbiorze testowym dla poszczególnych poziomów kompresji. Wartości te mogą się nieznacznie różnić w zależności od liczby epok treningowych oraz inicjalizacji wag początkowych sieci.

Tabela 1: Wpływ wymiaru przestrzeni ukrytej na błąd MSE

Wymiar ukryty (latent_dim)	Stopień kompresji	Średni błąd MSE (Test Loss)
10	18.7×	0.004497
16	11.68×	0.003786
40	4.67×	0.001928
100	1.87×	0.001066
128	1.46×	0.000358



Rysunek 1: Zależność błędu rekonstrukcji MSE od wielkości wąskiego gardła o rozmiarze 40 dla przykładowej serii testowej.

4.2 Wnioski i interpretacja

Na podstawie przeprowadzonych badań sformułowano następujące wnioski:

- **Monotoniczna zależność błędu:** Wyniki jednoznacznie potwierdzają teoretyczne założenia działania autoenkoderów. Zmniejszanie rozmiaru wąskiego gardła (ang. *bottleneck*) z 128 do 10 wymiarów powoduje stopniowy, nieliniowy wzrost błędu średniokwadratowego z poziomu 0.000358 do 0.004497.
- **Granica użyteczności diagnostycznej:** Dla modeli o wymiarach ukrytych równych 40 oraz 100, rekonstrukcja sygnału EKG zachowuje bardzo wysoką wierność, odwzorowując precyzyjnie kluczowe załamki (w tym wysokie piki kompleksu QRS). Zejście do skrajnych wartości kompresji (`latent_dim = 16` oraz `latent_dim = 10`), generujących blisko dwudziestokrotne zmniejszenie objętości danych ($18.7\times$), skutkuje zauważalnym zniekształceniem oraz odfiltrowaniem drobniejszych cech morfologicznych sygnału.
- **Punkt optymalny (Trade-off):** Analiza inżynierska wskazuje `latent_dim = 40` jako optymalny punkt kompromisu. Zapewnia on ponad 4.5-krotną redukcję pasma i pamięci przy jednoczesnym utrzymaniu błędu MSE na niskim, bezpiecznym poziomie (poniżej 0.002).

5 Instrukcja replikacji wyników

Aplikacja została zaprojektowana w sposób modułowy, co pozwala na łatwe odtworzenie eksperymentów:

1. **Trening modelu:** `python training.py 40 10`
2. **Uruchomienie kompresji:** `python compressor.py compress ekg.npy compressed.pkl 40 autoencoder_latent_40.pth`
3. **Dekompresja sygnału:** `python compressor.py decompress compressed.pkl restored.npy 40 autoencoder_latent_40.pth`
4. **Generowanie wykresów:** `python visualize.py 40`